System Architecture Documentation Pipeline

Step 1: High-Level System Context & Containers (The Bird's-Eye View)

Before diving into the code, you need to establish how your system interacts with the outside world and what its major moving parts are.

- **Detailed Explanation:** Create a Level 1 (System Context) and Level 2 (Container) diagram based on the C4 model. You need to map out your users (e.g., Admins, Customers), your core application (the Modular Monolith), and external dependencies (Stripe API, Elasticsearch, Postgres, Redis). This answers *what* the system is and *who* uses it.
- **Tools Required: Structurizr** (native C4 support), **Draw.io**, or **Excalidraw** (for quick, collaborative whiteboarding).

Step 2: Modular Monolith Boundaries & Internal Architecture

Because you are building a modular monolith, strictly defining your domain boundaries is the most critical part of your design.

- **Detailed Explanation:** Document the internal modules (e.g., User Management, Billing/Payments, Search Sync). Explain the rules of engagement: how do modules communicate? Typically, synchronous reads might happen via direct method calls across module boundaries, while state mutations should trigger domain events.
- **Tools Required: Mermaid.js** or **PlantUML**. These allow you to write diagrams as code, which makes updating them as your modules evolve frictionless.

Step 3: Database Schema & Data Modeling

Your application state lives in PostgreSQL, but how it's structured defines your system's efficiency.

- **Detailed Explanation:** Create an Entity-Relationship Diagram (ERD). Since this is a modular monolith, clearly delineate which tables belong to which module. A strict modular monolith rule is that **Module A cannot directly query Module B's database tables**; it must ask Module B for the data via an interface. Document your primary keys, foreign keys, and indexes.
- **Tools Required: dbdiagram.io** (excellent for writing DB schemas as code), **DataGrip**, or **Prisma Studio** (if you are using Prisma as your ORM).

Step 4: Event-Driven Workflows & Async Processing

You have Redis (BullMQ), Elasticsearch, and Stripe Webhooks. This means a lot of things happen asynchronously in the background.

- **Detailed Explanation:** Map out the exact flow of data for asynchronous operations using Sequence Diagrams. For example, detail the Stripe Webhook flow:
1. Stripe sends a payment_intent.succeeded webhook.
2. The Monolith validates the event and updates Postgres.
3. The Monolith dispatches a PaymentProcessed event to BullMQ.
4. A background worker picks up the job and indexes the updated user status into Elasticsearch.
- **Tools Required: Mermaid.js** (Sequence Diagrams) or **WebSequenceDiagrams**.

Step 5: Security, Authentication & Authorization

Security cannot be an afterthought; it must be documented explicitly before implementation.

- **Detailed Explanation:** Document the JWT lifecycle—where is it generated, what claims does it contain, how is it validated, and what is the refresh token strategy? Next, create an RBAC (Role-Based Access Control) matrix. This is usually a simple table mapping roles (e.g., SuperAdmin, Manager, User) to system resources and actions (Create, Read, Update, Delete).
- **Tools Required: Markdown Tables** for the RBAC matrix. **Swagger/OpenAPI** to define which endpoints require which JWT scopes.

Step 6: API Contracts

Frontend developers or external consumers need to know how to talk to your system.

- **Detailed Explanation:** Define the exact inputs, outputs, and HTTP status codes for your REST or GraphQL APIs. This serves as a contract between the backend and frontend teams, allowing them to work in parallel.
- **Tools Required: Swagger/OpenAPI specification**. Tools like **Postman** or **Insomnia** can also be used to share executable API specs.

Where to Store the Documentation?

The industry standard has shifted away from isolated Word documents or messy Google Drives toward **Docs-as-Code** and Developer Portals.

1. **Docs-as-Code (The Baseline):** Store your documentation in a /docs folder directly inside your Git repository using Markdown files. This ensures your documentation is version-controlled alongside your code. When a PR updates the system, the PR should also update the docs.
2. **Centralized Wikis (The Standard):** Sync your markdown files to **Confluence** or **Notion**.

This makes the architecture accessible to non-technical stakeholders (like Product Managers) who don't browse GitHub.

3. **Internal Developer Portals (Enterprise Scale):** If your engineering team is large, tools like **Spotify's Backstage** are the gold standard. They aggregate API specs, CI/CD statuses, and architecture diagrams into one searchable hub.

To get started, which module of the system (e.g., Stripe payments, Elasticsearch syncing, or User Auth) do you feel will be the most complex to design first?